

占城大师开发方式与项目结构

PDF 生成日期：2026-07-05 | 源文件：docs/DEVELOPMENT_WORKFLOW.md

本文档是 zhanchengdashi 的日常开发入口，也是公共《通用游戏开发流程文档》在本项目里的适配层。

公共流程上游：

- C:\Users\76398\Documents\Codex\2026-07-03\codex-game-studio-default\outputs\codex-game-studio-general-game-development-process.md

使用方式：公共流程负责角色路由、生命周期、专项流程、任务卡和 Definition of Done；本项目文档负责把这些规则落到 zhanchengdashi 的目录结构、Godot/GDScript、配置表、验证脚本和 GitHub 同步上。后续开发必须优先遵循公共流程，再按本项目约定执行。

0. 当前项目流程确认

- 上游流程：使用公共 codex-game-studio-general-game-development-process.md。
- 项目适配：使用本文档 docs/DEVELOPMENT_WORKFLOW.md。
- 项目阶段：当前属于 Prototype / Vertical Slice 之间，已经有 Godot 可运行原型、卡牌/抽卡/战斗/编组闭环和配置表基础。
- 默认引擎角色：公共流程中的 Engine Specialist 在本项目映射为 Godot Specialist。
- 默认语言角色：公共流程中的 Language Specialist 在本项目映射为 GDScript Specialist。
- 文档先行：以后所有玩法、数值、UI、系统或技术结构修改，都必须先更新对应设计/流程文档，再实装到游戏中。
- 策划案制图门禁：玩法流程图和 UI/UE 图必须随策划案交付，正式源文件使用 diagrams.net/draw.io、Figma、FigJam、Axure、Adobe XD、Illustrator 等专业工具格式；Mermaid 只能作为草稿或文档内辅助说明。
- 版本纪律：每次完成修改必须验证、提交 Git，并推送到 GitHub。

1. 开发原则

- 所有任务默认按游戏开发任务处理，采用 Codex Game Studio 的角色视角：制作、玩法、技术、美术/UI、Godot、GDScript、QA。
- 玩法数值优先数据驱动：可配置的设计值进入 [config/tables/](#)，运行时 JSON 由 [tools/export_config.py](#) 输出到 [runtime/config/](#)。
- 原型表现优先使用引擎内程序化反馈：Tween、缩放、位移、闪烁、粒子、材质调色和 UI 弹跳，避免过早制作序列帧资源。
- UI 调整遵循移动端街机风格：粗描边、硬阴影、高饱和按钮、明确进度条、可扫描信息层级。
- 不复制商业游戏的名称、角色、图标、货币、布局、字体或专有资产，只借鉴抽象方向。
- 文档是实现前置条件：先在 [design/](#) 或 [docs/](#) 中记录要改的规则、数值、流程、界面或技术决策，再修改配置、脚本、场景和资源。
- 每次完成修改都必须提交 Git，并同步到 GitHub 远端。

2. 标准任务流程

1. 读上游：确认公共流程中对应的角色路由、专项流程和 Definition of Done。
2. 读项目：看 [AGENTS.md](#)、本文档、相关 [docs/](#)、当前文件和 [git status](#)。
3. 定职责：按任务类型使用最小必要角色组合，例如 UI 走 [Art Director](#) -> [UI Programmer](#) -> [Godot Specialist](#) -> [QA Lead](#)。
4. 文档先行：先更新对应文档。玩法/数值/UI 进入 [design/](#)，工程流程/结构进入 [docs/](#)，必要时同步生成 PDF。
5. 专业制图：策划案必须补齐玩法流程图和 UI/UE 图；源文件放 [docs/diagrams/](#)，预览图放 [output/diagrams/](#)，PDF 必须嵌入或引用图件。
6. 定落点：再判断改配置、脚本、场景、资源或工具，避免把设计值写死在代码里。
7. 小步实现：按已经更新的文档实装，保持提交范围聚焦，沿用现有脚本、绘制和数据结构。
8. 本地验证：按改动类型运行对应检查，Godot 脚本改动必须启动项目确认无解析错误。
9. 整理差异：查看 [git diff --check](#) 和 [git diff --stat](#)，确认没有无关破坏。
 10. 提交同步：[git add](#)、[git commit](#)、[git push origin main](#)。
 11. 回报结果：说明先改了哪份文档、实装改了什么、验证了什么、提交号和远端同步状态。

3. 公共流程阶段映射

公共阶段	本项目状态	本项目执行方式
0. Project Startup	已完成	Git、README、目录、Godot 入口、配置表基础已建立
1. Concept	已有初版	当前方向是移动端卡牌+占地战斗原型，继续在 docs/CURRENT_GAME_DESIGN.md 沉淀
2. Prototype	进行中	用 Godot 快速验证战斗、抽卡、编组、升级、地块和 UI
3. System Design	进行中	规则和数值优先进入文档与配置表，脚本只做运行实现
3A. Data Config	已建立并持续扩展	config/tables/ -> tools/validate_config.py -> tools/export_config.py -> runtime/config/

公共阶段	本项目状态	本项目执行方式
4. Technical Architecture	需要持续补强	当前集中在 <code>scripts/app/main.gd</code> ，复杂度上升后拆分到 <code>scripts/app/ui/</code> 、 <code>battle/</code> 、 <code>cards/</code>
5. Vertical Slice	进行中	目标是打通大厅、编组、抽卡、战斗、结算、成长的完整闭环
6. Implementation	按任务推进	每个功能用小步提交推进，保持可运行
7. QA and Tuning	每轮执行	配置校验、GDScript 缩进、Godot 启动、必要时截图/手测
8. Milestone Review	阶段性执行	大功能后判断继续、重做、砍掉或调优
9. Version Finish	每次提交执行	验证通过、commit、push、记录 hash

4. 目录职责

路径	职责	使用规则
<code>config/tables/</code>	设计源表	卡牌、单位、经济、关卡、掉落、地块等可配置内容优先放这里
<code>runtime/config/</code>	运行时配置	由导出工具生成，只有作为引擎读取源时才提交
<code>scripts/</code>	Godot 脚本	放运行时代码、原型逻辑、UI 绘制、配置读取
<code>scenes/</code>	Godot 场景	放可运行场景、入口场景和可复用节点
<code>assets/</code>	源资产	美术、音频、UI、卡牌图、建筑图和后续特效资源
<code>tools/</code>	开发工具	校验、导出、文档生成、批处理和 QA 辅助脚本
<code>tests/</code>	自动化测试	放配置、规则、生成器和核心公式测试
<code>docs/</code>	可读文档	设计说明、流程、UI/UX、平衡、结构约定
<code>docs/diagrams/</code>	专业制图源文件	放 draw.io、Figma/FigJam 导出源、Axure、XD、AI 等可继续编辑的图件源
<code>output/diagrams/</code>	图件预览	放 PNG/SVG/PDF 预览，供策划案和评审 PDF 引用
<code>output/pdf/</code>	审阅版文档	文档 PDF 输出，便于给人查看和归档
<code>tmp/</code>	临时产物	本地临时文件，不提交

5. 改动落点矩阵

需求类型	首选落点	必要验证
卡牌/单位/经济/掉落数值	先改 <code>design/</code> 或配置说明，再改 <code>config/tables/</code>	<code>tools/validate_config.py</code> ，必要时 <code>tools/export_config.py</code>
战斗规则和交互逻辑	先改 <code>design/</code> ，再改 <code>scripts/app/</code> 或相关 Godot 脚本	GDScript 缩进检查，Godot 启动
UI 布局和绘制	先改 UI/UE 设计文档和 <code>docs/diagrams/</code> 图件，再改 <code>scripts/app/</code> 、 <code>assets/</code>	Godot 启动，必要时截图人工检查
新资源或美术方向	先改美术方向文档，再改 <code>assets/</code>	资源能加载，路径不硬编码到错误位置
设计决策和流程	<code>docs/</code>	生成 PDF，检查可读性
校验或导出能力	<code>tools/</code> 、 <code>.github/workflows/</code>	本地运行工具，确认 CI 入口可用

6. 验证门禁

- 任意 .gd 修改后运行 `tools/check_gd_indentation.py`，并启动 Godot 项目确认没有 parser error。
- 任意配置表修改后运行 `tools/validate_config.py`；如果运行时 JSON 需要更新，再运行 `tools/export_config.py`。
- Godot 在 Windows 出现 应用程序错误、内存不能为 read 或启动即崩溃时，优先检查渲染后端；本项目默认不强制 D3D12，`project.godot` 应使用 Vulkan 作为 Windows 默认渲染驱动，只有在专门兼容性测试通过后才恢复 D3D12。
- `config/tables/` 下的 CSV 必须保持 UTF-8 或 UTF-8 BOM 编码；不要提交 GBK/ANSI 表格。若 `tools/validate_config.py` 报 Unicode decode 错误，先转码源 CSV，再导出 `runtime/config/`。
- 任意文档交付后生成 PDF 到 `output/pdf/`，并渲染检查页面是否可读、无重叠、无截断。
- 任意策划案交付必须检查玩法流程图和 UI/UE 图是否存在专业源文件、预览图和 PDF 可读版本。
- 提交前运行 `git diff --check`，确认没有尾随空白或格式错误。
- 推送前确认 `git status --short --branch`，避免遗漏文件。

7. Godot 与 GDScript 约定

- 本项目 .gd 使用 tab 缩进，规则写在 `.editorconfig`。
- 不做盲目的 tab/space 全局替换；修复缩进时先保护代码块层级。
- 运行错误优先从 Godot parser/debug 输出定位，再修代码。
- UI 原型当前以 `scripts/app/main.gd` 的绘制函数为主，新增抽象时必须避免让单文件继续无边界膨胀；可复用逻辑成熟后再拆分模块。

8. Git 与 GitHub 同步

- 工作区可能已有他人或前序未提交改动；先确认来源和范围，不回滚无关修改。
- 提交保持聚焦，但如果用户要求“每次修改都同步”，完成后必须提交并推送。
- 默认远端分支为 `main`，推送命令为 `git push origin main`。
- 每次最终回报包含提交号、验证结果和工作区是否干净。

9. 后续结构演进方向

- 当 `scripts/app/main.gd` 继续增长时，优先拆出 `scripts/app/ui/`、`scripts/app/battle/`、`scripts/app/cards/`、`scripts/app/config/`。
- 将硬编码原型参数逐步迁移到 `config/tables/`，保留脚本中的常量只作为临时桥接。
- 为核心规则补 `tests/`：卡牌升级、抽卡概率、地块生成、战斗结算和配置导出。
- UI 反复修改后沉淀 `docs/UI_COMPONENT_GUIDE.md`，把卡牌、资源条、地块提示、底部导航做成可复用规范。